

Le Projet : RT-THM (Real-Time Task & Health Monitor)

L'objectif est de créer un système de surveillance de santé de processus. Un **Processus Parent (Superviseur)** gère plusieurs **Processus Fils (Workers)** qui effectuent des tâches simulées. Le superviseur doit réagir en temps réel si un fils "tombe" ou devient instable.

1. Fonctionnalités clés :

- **Gestion Multi-Processus** : Utilisation de `fork()` pour créer des tâches indépendantes.
- **Communication Inter-Processus (IPC)** : Utilisation de **Mémoire Partagée** (`shmget/shmat`) pour que les fils écrivent leur état et que le parent le lise.
- **Signaux Temps Réel** : Utilisation de `SIGUSR1` ou `SIGALRM` pour notifier le parent d'un événement critique.
- **Ordonnancement** : Utilisation de `nanosleep()` pour simuler des contraintes temporelles strictes.
- **Récupération (Healing)** : Si un fils meurt, le parent doit le détecter via `waitpid()` et le relancer automatiquement.

Architecture du Code (Structure)

Le Code Source (Squelette de base)

Voici une version simplifiée pour vous lancer. Ce code illustre la création, la communication via mémoire partagée et la surveillance.

```
#include <stdio.h>

#include <stdlib.h>

#include <unistd.h>

#include <sys/types.h>

#include <sys/ipc.h>

#include <sys/shm.h>

#include <sys/wait.h>

#include <signal.h>

#include <time.h>

#define MAX_WORKERS 3
```

```
#define SHM_SIZE 1024

// Structure pour l'état des processus en mémoire partagée
typedef struct {
    int pid;
    int health_score;
    int tasks_completed;
} ProcessStat;

// Variable globale pour accéder à la mémoire partagée
ProcessStat *shared_stats;

void handle_sigint(int sig) {
    printf("\n[SUPERVISEUR] Nettoyage et arrêt du système...\n");
    shmdt(shared_stats);
    exit(0);
}

void worker_process(int id) {
    srand(time(NULL) ^ (getpid() << 16));
    printf("[WORKER %d] Démarré (PID: %d)\n", id, getpid());

    while (1) {
        // Simuler un travail

        int work_time = (rand() % 3) + 1;
        sleep(work_time);

        // Mettre à jour les stats en mémoire partagée
    }
}
```

```

    shared_stats[id].pid = getpid();

    shared_stats[id].tasks_completed++;

    shared_stats[id].health_score = rand() % 100;


    // Simulation d'un crash aléatoire (5% de chance)
    if (shared_stats[id].health_score < 5) {
        printf("[WORKER %d] Erreur critique ! Crash...\n", id);
        exit(EXIT_FAILURE);
    }
}

}

int main() {
    int shmid;

    signal(SIGINT, handle_sigint);


    // 1. Création de la mémoire partagée

    shmid = shmget(IPC_PRIVATE, sizeof(ProcessStat) * MAX_WORKERS, IPC_CREAT |
0666);

    shared_stats = (ProcessStat *)shmat(shmid, NULL, 0);


    pid_t pids[MAX_WORKERS];


    // 2. Lancement des Workers
    for (int i = 0; i < MAX_WORKERS; i++) {
        pids[i] = fork();

        if (pids[i] == 0) {
            worker_process(i);

```

```

    }
}

// 3. Boucle de Supervision (Temps Réel)
printf("[SUPERVISEUR] Surveillance active...\n");
while (1) {
    printf("\n--- Dashboard Temps Réel ---\n");
    for (int i = 0; i < MAX_WORKERS; i++) {
        printf("Worker %d | PID: %d | Santé: %d%% | Tâches: %d\n",
            i, shared_stats[i].pid, shared_stats[i].health_score,
shared_stats[i].tasks_completed);

        // Vérifier si le processus est vivant
        int status;
        pid_t result = waitpid(pids[i], &status, WNOHANG);
        if (result != 0) {
            printf("[ALERTE] Worker %d est mort. Relance en cours...\n", i);
            pids[i] = fork();
            if (pids[i] == 0) worker_process(i);
        }
    }
    sleep(2); // Fréquence de rafraîchissement du moniteur
}

return 0;
}

```

Pour transformer ce code en un projet de niveau professionnel, vous devriez ajouter :

1. **Synchronisation (Sémaphores)** : Actuellement, le code écrit dans la mémoire partagée sans verrou. Utilisez `semget` et `semop` pour éviter les "Race Conditions".
2. **Fichier de Configuration** : Permettre de définir le nombre de processus et leurs priorités via un fichier `.conf` ou `.json`.
3. **Logs de performance** : Chaque événement (crash, redémarrage, tâche finie) doit être écrit dans un fichier de log avec un timestamp précis (microsecondes).
4. **Interface Ncurses** : Au lieu de simples `printf`, utilisez la bibliothèque `ncurses` pour créer un véritable tableau de bord graphique dans le terminal.
5. **Gestion des signaux** : Implémenter une communication où le parent peut envoyer un signal `SIGUSR1` pour demander à un fils de se mettre en "mode économie d'énergie" (ralentir sa boucle).

Les détails de chaque partie

1. Synchronisation par Sémaphores (Sécurité des données)

Dans un système multi-processus, si le parent lit la mémoire partagée pendant qu'un fils écrit, vous obtenez des données corrompues. Les sémaphores agissent comme des feux de signalisation.

- **Le concept** : On utilise un sémaphore binaire (Mutex). Avant d'écrire, le fils fait un P (wait). Une fois fini, il fait un V (signal).
- **Implémentation** : Utilisez `semget()` et `semop()`.
- **Pourquoi c'est vital** : Cela garantit l'**atomicité** des opérations sur votre tableau de bord.

2. Configuration via Fichier (Flexibilité)

Au lieu de coder en dur `MAX_WORKERS`, votre programme doit lire un fichier au démarrage (ex: `config.txt`).

- **Format suggéré** :

Ini, TOML

`workers=5`

`refresh_rate=500ms`

`log_level=DEBUG`

- **Technique C** : Utilisez `fopen()`, `fgets()` et `strtok()` pour parser les lignes. Cela permet de tester différentes charges système sans recompiler le code.

3. Journalisation (Logging) avec Horodatage Précis

En temps réel, savoir qu'une erreur est arrivée "à 14h" n'est pas suffisant. Il faut de la précision à la microseconde.

- **Fonction de Log :** Créez une fonction `log_event(char* message)` qui :
 1. Récupère le temps avec `gettimeofday()` ou `clock_gettime(CLOCK_REALTIME)`.
 2. Ajoute le PID du processus concerné.
 3. Écrit dans un fichier `system.log`.
- **Exemple de sortie :** `[2026-04-15 12:45:01.004231] [PARENT] Worker 2 (PID 4562) restarted after crash.`

4. Interface Ncurses (Visualisation Professionnelle)

L'interface standard (terminal) défile trop vite. `ncurses` permet de diviser votre écran en "fenêtres" fixes.

- **Structure de l'écran :**
 - **Haut :** État global du système (CPU, mémoire totale utilisée par le projet).
 - **Milieu :** Tableau fixe avec les statistiques de chaque Worker (Santé, Tâches).
 - **Bas :** Console de logs qui défile en direct.
- **Avantage :** Cela transforme votre projet de "petit script" en une véritable "application de monitoring".

5. Communication bidirectionnelle (Signaux & IPC)

Le superviseur ne doit pas juste regarder, il doit diriger.

- **Signaux Personnalisés :**
 - `SIGUSR1` : Le parent ordonne au fils de suspendre son travail (Pause).
 - `SIGUSR2` : Le parent ordonne au fils de reprendre (Resume).
- **Mémoire Partagée de Commande :** Ajoutez un champ `command` dans votre structure `ProcessStat`. Le fils lit ce champ à chaque boucle. Si le parent écrit 1, le fils s'arrête proprement.

Structure de Dossier Recommandée

Pour un projet consistant, l'organisation est la clé :

/RT-THM-Project

```
├── src/
|   ├── main.c    # Superviseur
|   ├── worker.c  # Logique des fils
|   ├── ipc_utils.c # Gestion SHM, Sémaphores
|   └── logger.c   # Gestion des fichiers logs
├── include/
|   ├── project.h  # Structures et prototypes
|   └── config.h
├── config.txt     # Fichier de config
└── Makefile       # Pour compiler le tout proprement
```

Partie 6 : Conformité MISRA C et Analyse Statique

L'objectif ici n'est pas seulement d'écrire du code qui fonctionne, mais du code **sûr, prévisible et maintenable**. Comme les règles MISRA sont très strictes (plus de 140 règles), nous allons nous concentrer sur les plus cruciales pour la gestion de processus.

1. Les Règles MISRA Clés pour votre Projet

Pour ce projet en C, vous devrez surveiller particulièrement ces points :

- **Règle 17.7 (Valeurs de retour) :** La valeur de retour de toute fonction non-void (comme shmget, fork, semop) **doit** être testée. Ne jamais supposer qu'un appel système réussit.
- **Règle 21.3 (Mémoire dynamique) :** L'utilisation de malloc, free, realloc est **interdite** après l'initialisation. Pour votre projet, l'allocation de la mémoire partagée doit se faire une seule fois au démarrage.
- **Règle 11.4 (Conversion de pointeurs) :** MISRA décourage la conversion de pointeurs en entiers. Cependant, pour l'IPC, c'est parfois nécessaire. Il faut alors utiliser des "wrappers" sécurisés et documentés.

- **Règle 2.2 (Code mort) :** Tout code qui n'influe pas sur le comportement du programme doit être supprimé.

2. Le Script d'Interprétation et d'Audit (Python/Bash)

Pour générer un fichier "interprétant" ou auditant le code, nous allons créer un script compagnon (en Python) qui analyse votre source C et génère un rapport de conformité.

Ce que fait le script :

1. Il scanne les fichiers .c pour vérifier la présence de fonctions interdites ou de mauvaises pratiques.
2. Il vérifie si chaque fork() est suivi d'une vérification d'erreur.
3. Il génère un fichier audit_report.txt.

Exemple de script d'audit (checker.py) :

```
import re
import os

def check_misra_basics(file_path):
    report = []
    with open(file_path, 'r') as f:
        content = f.readlines()

    for i, line in enumerate(content):
        # Vérification Rule 17.7 : Appel système sans vérification immédiate
        if "fork()" in line and "pid =" not in line:
            report.append(f"Ligne {i+1}: ALERTE MISRA - Valeur de retour de fork() non capturée.")

        # Vérification Rule 21.3 : Allocation dynamique interdite en runtime
        if "malloc(" in line:
            report.append(f"Ligne {i+1}: ALERTE MISRA - malloc() détecté. Préférez l'allocation statique ou initiale.")
```



```

return report

def generate_report(results):
    with open("misra_interpretation.txt", "w") as f:
        f.write("=== RAPPORT D'INTERPRÉTATION ET DE CONFORMITÉ ===\n")
        if not results:
            f.write("Félicitations : Aucune violation majeure détectée.\n")
        else:
            for error in results:
                f.write(error + "\n")

# Exécution
violations = check_misra_basics("src/main.c")
generate_report(violations)

```

3. Génération du fichier "Interpréteur de Code" (Documentation)

Pour que votre projet soit complet, vous devez générer automatiquement une documentation qui explique la logique de chaque bloc. On utilise souvent **Doxygen** pour cela.

- **Action** : Ajoutez des commentaires formatés `/** ... */` au-dessus de chaque fonction.
- **Résultat** : Une commande simple générera un fichier HTML ou PDF interprétant chaque module, chaque structure de données et chaque flux de signal.

Résumé de l'architecture finale du projet :

1. **Cœur (C)** : Multi-processus synchronisés par sémaphores.
2. **Interface (Ncurses)** : Tableau de bord visuel.
3. **Audit (Python)** : Vérificateur de règles MISRA.
4. **Rapport (Txt)** : Fichier d'interprétation généré après chaque compilation.